

LAB ASSIGNMENT: Hardware Timer Delegation Across M-Mode, S-Mode, and U-Mode

Module: Privilege Isolation, CSR Delegation, and CLINT Timer Preemption

Hardware Peripherals: Core Local Interruptor (CLINT), `mtime`, `mtimecmp`

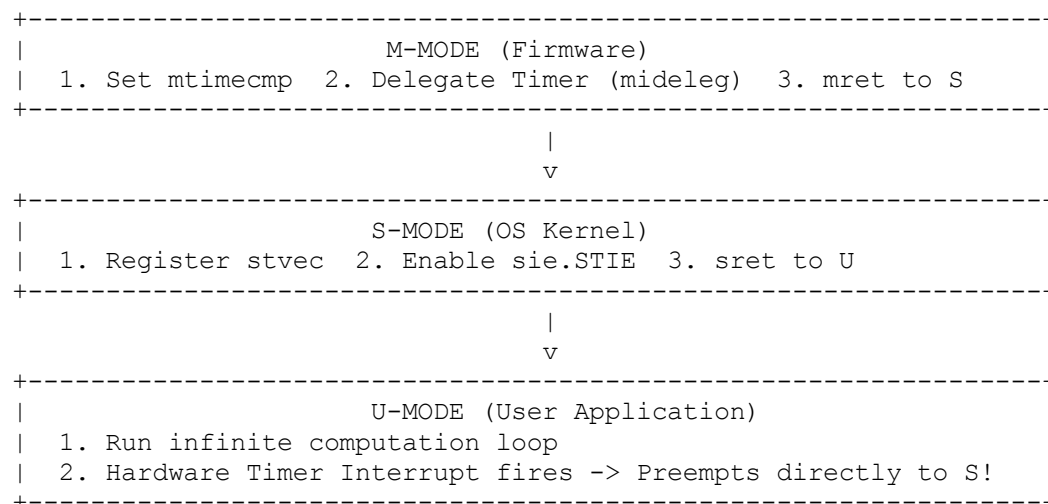
1. Executive Summary

In a modern multi-tier operating system architecture (such as Linux on RISC-V), **Machine Mode (M-Mode)** acts as secure hardware, **Supervisor Mode (S-Mode)** executes the OS kernel, and **User Mode (U-Mode)** runs unprivileged user applications.

To prevent firmware overhead on every timer tick, M-Mode **delegates** timer interrupts down to S-Mode using the `mideleg` Control and Status Register (CSR). When delegated, a CLINT timer interrupt firing during U-Mode execution bypasses M-Mode entirely and traps directly into the **S-Mode OS Kernel trap handler** (`stvec`).

In this project, you will configure timer delegation in M-Mode, transition through S-Mode down to U-Mode, run a user computation loop, and verify direct preemption into S-Mode.

2. Architecture Privilege Hierarchy & CSRs Used



Core Control Registers

- **`mideleg` (Machine Interrupt Delegation):** Bit 5 (`STIE`) delegates Supervisor Timer Interrupts to S-Mode.
- **`sstatus` / `sie` / `sip`:** Supervisor status, interrupt enable, and interrupt pending registers.

- **stvec / sepc / scause:** Supervisor trap vector, exception program counter, and cause registers.

3. Project Objectives

By completing this lab assignment, you will demonstrate the ability to:

1. Program the memory-mapped CLINT timer (`mtimecmp`). (**MMIO vs Port Mapped IO ?**)
2. Configure 3-level privilege transitions (**M** $\square$ **S** $\square$ **U**) using `mret` and `sret`.
3. Delegate hardware interrupts from M-Mode to S-Mode using `mideleg`.
4. Capture and handle an asynchronous Supervisor Timer Trap (`scause == 0x8000000000000005`) directly inside S-Mode.

4. Step-by-Step Lab Tasks

Task 1: M-Mode Firmware Setup & Interrupt Delegation

1. Read `mtime (0x0200BFF8)` and set `mtimecmp (0x02004000)` to `mtime + 100,000 ticks`.
2. Delegate Supervisor Timer Interrupts by setting bit 5 in `mideleg (mideleg.STIE)`.
3. Set `mepc = s_mode_kernel_entry`.
4. Set `mstatus.MPP = 01` (target: S-Mode) and execute `mret` to enter S-Mode.

Task 2: S-Mode OS Kernel Setup & Transition to U-Mode

1. Register the Supervisor trap handler address (`la t0, s_mode_trap_handler`) in `stvec`.
2. Enable Supervisor Timer Interrupts in S-Mode by setting bit 5 in `sie (sie.STIE)`.
3. Set `sepc = u_mode_user_app`.
4. Configure `sstatus`:
 - o Set `sstatus.SPP = 0` (target: U-Mode).
 - o Set `sstatus.SIE = 1` (enable global supervisor interrupts upon `sret`).
5. Execute `sret` to enter U-Mode.

Task 3: U-Mode Preemption & S-Mode Trap Verification

1. In `u_mode_user_app`, write an infinite loop that increments `x10`.
2. Verify that when `mtime >= mtimecmp`, execution jumps directly to `s_mode_trap_handler` (bypassing M-Mode).
3. Inspect `scause`:
 - o Confirm Bit 63 is 1 (Interrupt).
 - o Confirm Bits `[3:0]` equal 5 (Supervisor Timer Interrupt).

4. Clear the pending interrupt bit in `sip` or reset `mtimecmp` to `0xFFFFFFFFFFFFFFFF`.

5. Lab Report Questions

1. **Trap Delegation Architecture:** What is the primary performance reason for delegating timer traps to S-Mode via `mideleg` rather than catching every timer interrupt in M-Mode firmware?
2. **Interrupt Cause Codes:** Why is the interrupt cause code 5 (`scause`) when handled in S-Mode, whereas it was 7 (`mcause`) when handled in M-Mode?
3. **Privilege Isolation:** What happens if `u_mode_user_app` attempts to write directly to `stvec` or `mideleg`? What trap cause code is generated?